

НИУ ИТМО
Факультет ПИиКТ

Лабораторная работа №2
Вычислительная математика
Вариант №5

Преподаватель **Наумова Надежда Александровна**
Подготовил **Лоскутов Прохор Александрович**

Санкт - Петербург 2026

Оглавление

Оглавление	2
1 Вычислительная реализация задачи	3
1.1 Решение нелинейного уравнения	3
1.2 Решение системы нелинейных уравнений	6
2 Программная реализация задачи	10
2.1 Рабочие формулы и блок-схемы реализованных методов ..	10
2.2 Исходный код основных элементов расчёта	15
2.3 Интерфейс приложения	20
3 Выводы	20

1 Вычислительная реализация задачи

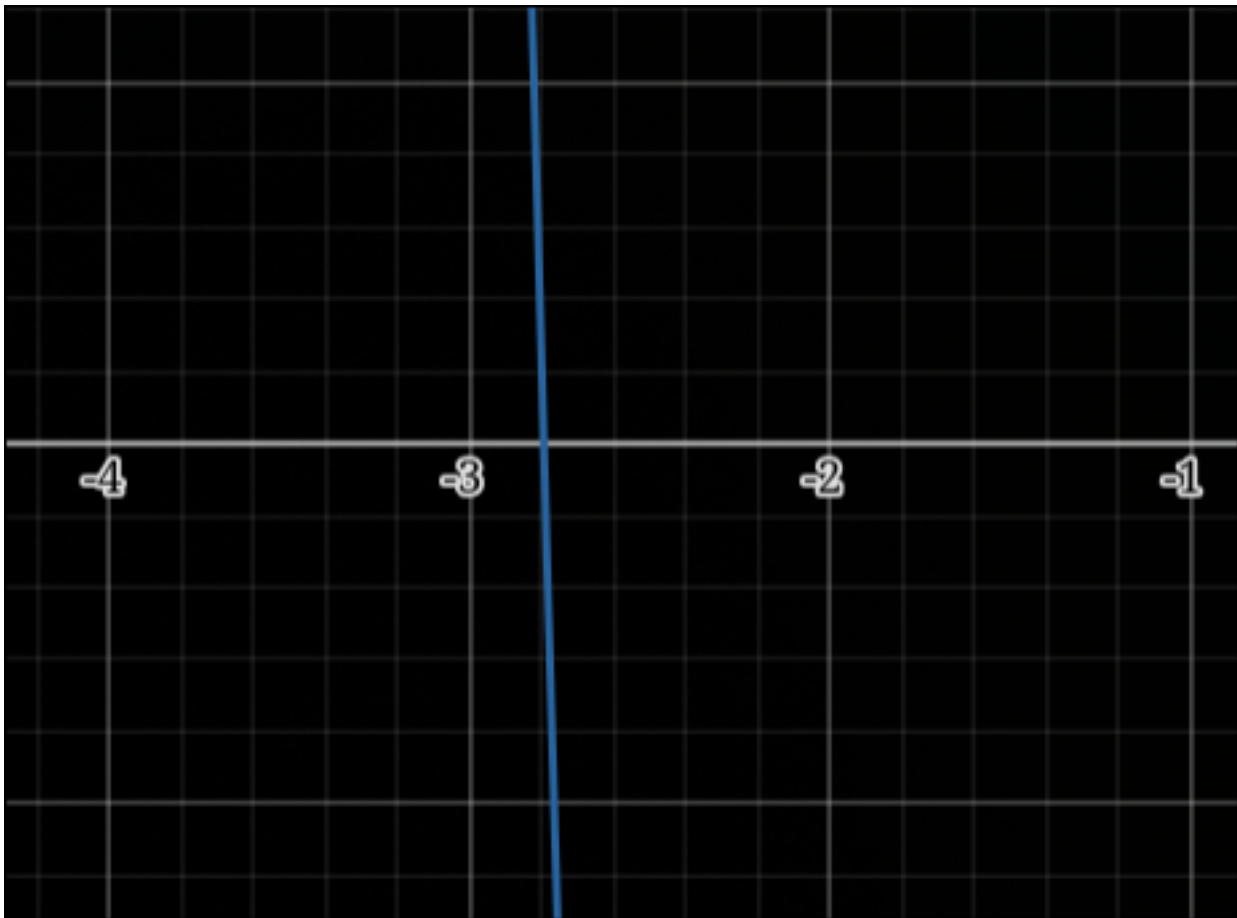
1.1 Решение нелинейного уравнения

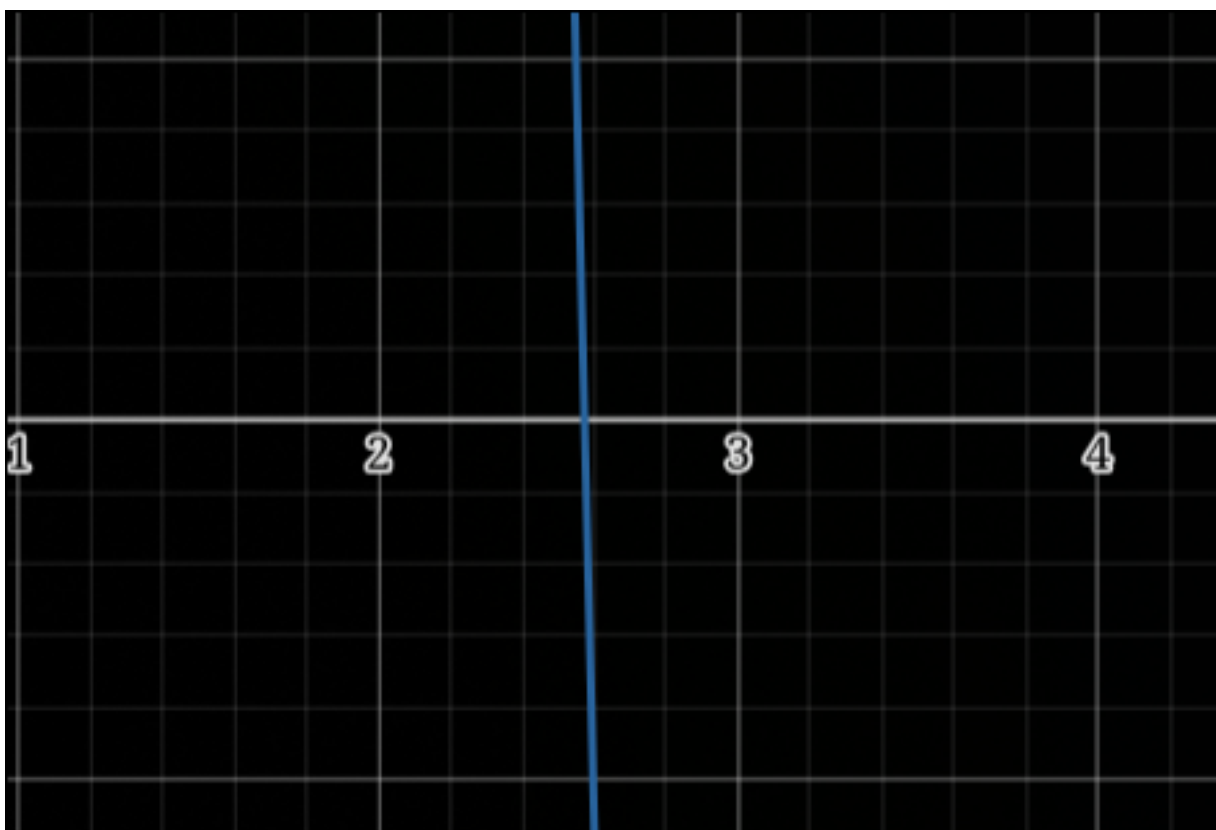
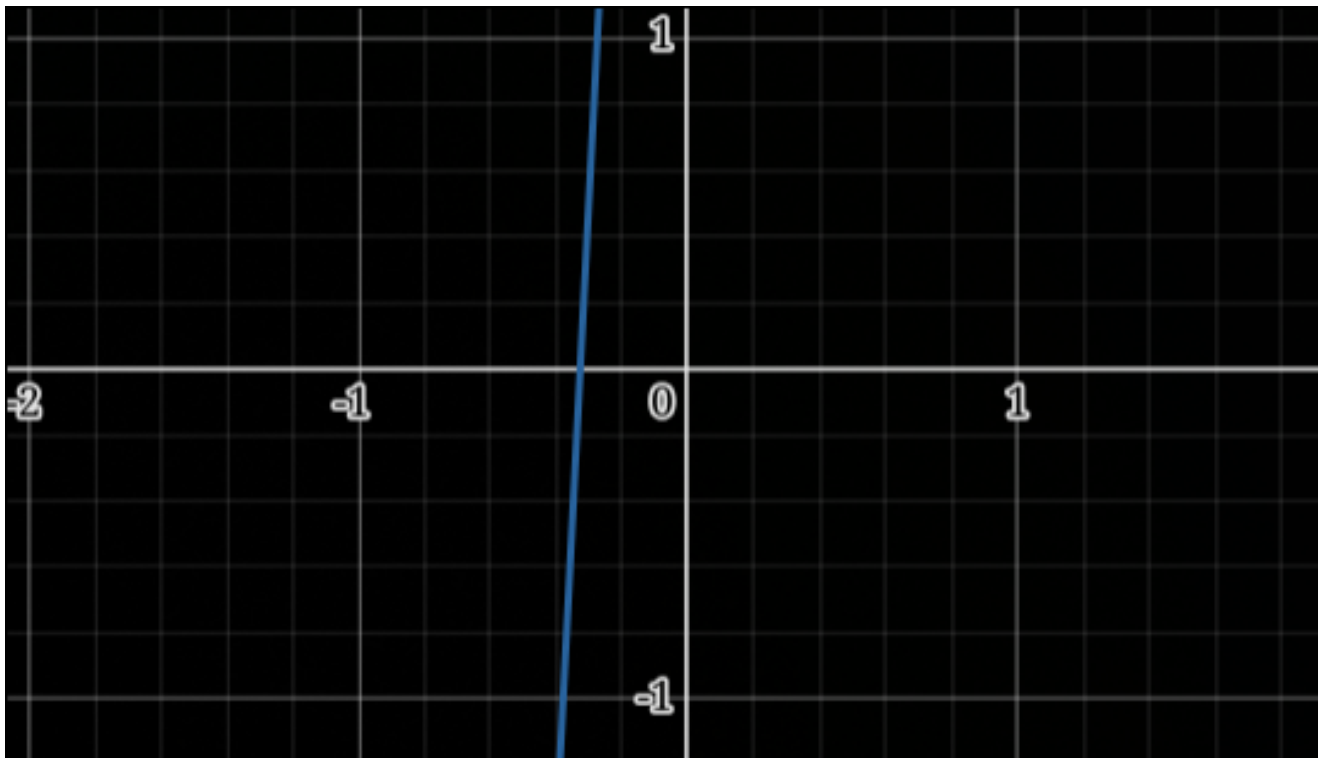
Дано нелинейное уравнение с тремя корнями:

$$y = -2.7 \cdot x^3 - 1.48 \cdot x^2 + 19.23 \cdot x + 6.35$$

1.1.1 Отделить корни заданного нелинейного уравнения графически

Построим графическое изображение уравнения в Desmos:





1.1.2 Определить интервалы изоляции корней

По изображениям, мы можем выделить три отрезка, в которых лежат наши корни:

$$x \in [-3; -2]$$

$$x \in [-1; 0]$$

$$x \in [2; 3]$$

1.1.3 Уточнить корни нелинейного уравнения с точностью $\varepsilon = 10^{-2}$.

В качестве методов нужно использовать:

Расположения корня	Метод уточнения
Левый	Простые итерации
Центральный	Секущие
Правый	Хорды

1.1.4 Таблицы с расчётами значений

Таблица 1

Уточнение корня уравнения методом простых итераций

№	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-3.000	-2.816	0.756	0.184
2	-2.816	-2.799	0.143	0.017
3	-2.799	-2.796	0.028	0.003
4	-2.796	-2.795	0.006	0.001

1.1.4.1 Проверка достаточного условия сходимости для итерационного метода.

$$f(x) = -2.7x^3 - 1.48x^2 + 19.23x + 6.35$$

$$f'(x) = -8.1x^2 - 2.96x + 19.23$$

Возьмём интервал локализации

$$[a, b] = [-3.0; -2.5]$$

На этом интервале

$$f'(x) < 0$$

Выберем

$$\lambda = \frac{1}{\max_{x \in [a, b]} |f'(x)|}$$

$$|f'(-3.0)| = 44.79, \quad |f'(-2.5)| = 24.645$$

$$\max_{x \in [a, b]} |f'(x)| = 44.79$$

$$\lambda = \frac{1}{44.79} \approx 0.0223$$

Итерационная функция:

$$\varphi(x) = x + \lambda f(x)$$

$$\varphi'(x) = 1 + \lambda f'(x)$$

Проверим условие сходимости на концах интервала:

$$\varphi'(-3.0) = 1 + 0.0223 \cdot (-44.79) \approx 0$$

$$\varphi'(-2.5) = 1 + 0.0223 \cdot (-24.645) \approx 0.450$$

Следовательно,

$$|\varphi'(x)| \leq q$$

$$q = \max_{x \in [a, b]} |\varphi'(x)| \approx 0.450 < 1$$

Достаточное условие сходимости выполняется, следовательно метод простой итерации сходится, **Q.E.D.**

Таблица 2

Уточнение корня уравнения методом секущих

№	x_{k-1}	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	-0.500	-0.499	-0.324	0.065	0.176
2	-0.499	-0.324	-0.327	-0.001	0.003

Таблица 3

Уточнение корня уравнения методом хорд

№	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ a - b $
1	2.000	3.000	2.438	17.290	-22.180	5.308	1.000
2	2.438	3.000	2.547	5.308	-22.180	1.134	0.562
3	2.547	3.000	2.569	1.134	-22.180	0.222	0.453
4	2.569	3.000	2.573	0.222	-22.180	0.043	0.431
5	2.573	3.000	2.574	0.043	-22.180	0.008	0.427

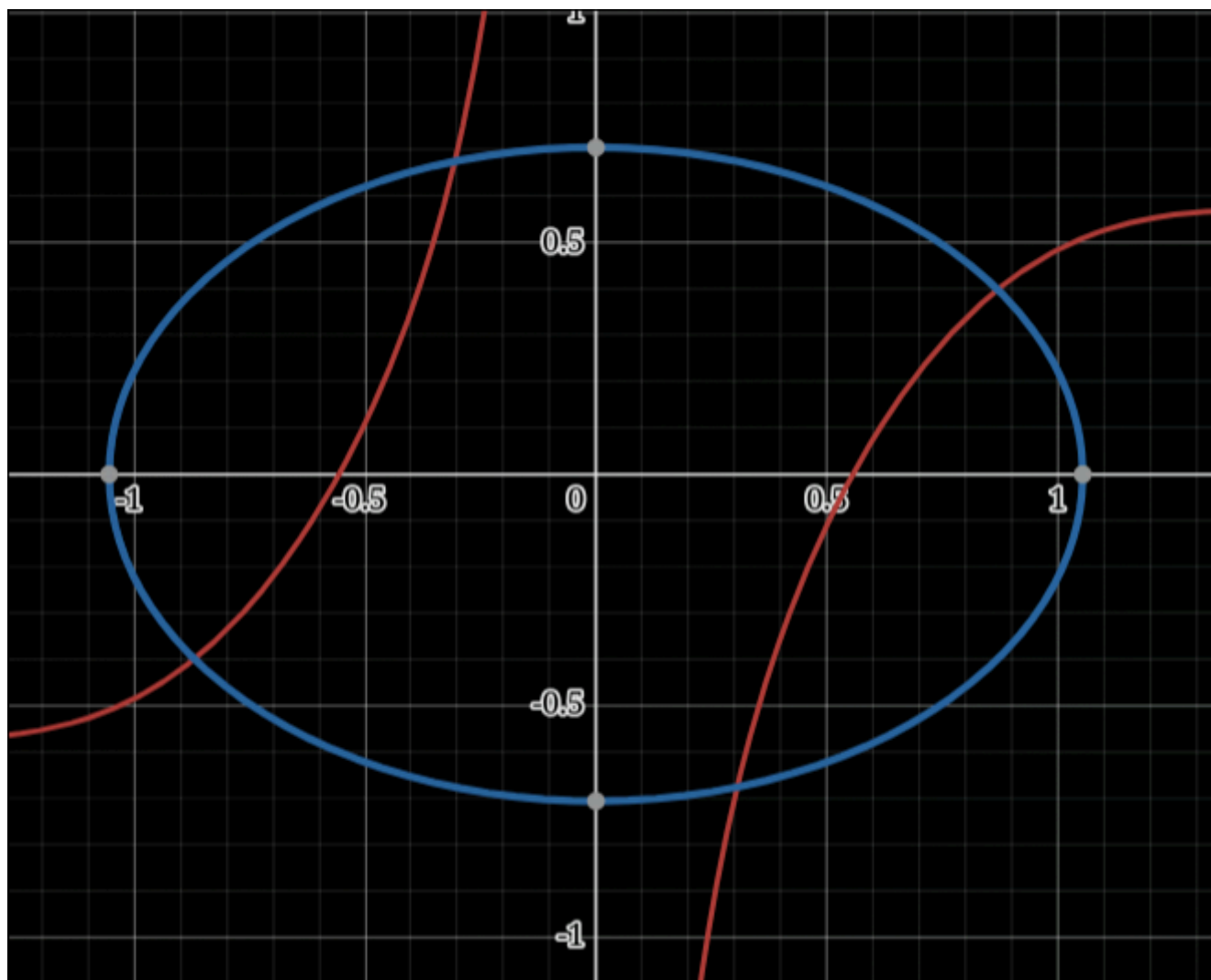
1.2 Решение системы нелинейных уравнений

1.2.1 Отделение корней заданной системы

По варианту:

Система	Метод
$\begin{cases} \tan(x \cdot y + 0.3) = x^2 \\ 0.9x^2 + 2y^2 = 1 \end{cases}$	Метод Ньютона

1.2.2 Графическое изображение



1.2.3 Анализ графика функции

Как видно из графика, система имеет 4 решения. Заметим, что первая функция – чётная, как и вторая. Это значит, что нам достаточно проанализировать две точки пересечения слева, а две – справа будут просто симметрией относительно центра, т.к. $f(-x, -y) = f(x, y)$.

Вот два отрезка, на которых мы будем уточнять корни.

$$x \in [-0.9; -0.8]$$

$$x \in [-0.4; -0.2]$$

1.2.4 Уточнение корней.

1. Приведём систему к виду $F(x, y) = 0$:

$$\begin{cases} \tan(x \cdot y + 0.3) = x^2 \\ 0.9x^2 + 2y^2 = 1 \end{cases} \rightarrow \begin{cases} f(x, y) = 0 \\ g(x, y) = 0 \end{cases} \rightarrow \begin{cases} \tan(x \cdot y + 0.3) - x^2 = 0 \\ 0.9x^2 + 2y^2 - 1 = 0 \end{cases}$$

Построим матрицу Якоби:

$$\frac{\partial f}{\partial x} = \frac{1}{\cos^2(x \cdot y + 0.3)} y - 2x, \quad \frac{\partial g}{\partial x} = 1.8x$$

$$\frac{\partial f}{\partial y} = \frac{1}{\cos^2(x \cdot y + 0.3)} x, \quad \frac{\partial g}{\partial y} = 4y$$

Решаем систему:

$$J(x, y) \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} = - \begin{pmatrix} f(x, y) \\ g(x, y) \end{pmatrix}$$

$$\begin{cases} \left(\frac{1}{\cos^2(x \cdot y + 0.3)} y - 2x \right) \Delta x + \left(\frac{1}{\cos^2(x \cdot y + 0.3)} x \right) \Delta y = x^2 - \tan(x \cdot y + 0.3) \\ 1.8x \Delta x + 4y \Delta y = 1 - 0.9x^2 - 2y^2 \end{cases}$$

1.2.4.1 Выберем начальное приближение: $x_0 = -0.9, y_0 = -0.4$.

На первой итерации система будет иметь вид:

$$\begin{cases} 1.159\Delta x - 1.442\Delta y = 0.034 \\ -1.620\Delta x - 1.600\Delta y = -0.049 \end{cases}$$

Решаем систему:

$$\begin{cases} 1.159\Delta x - 1.442\Delta y = 0.034 \\ -1.620\Delta x - 1.600\Delta y = -0.049 \end{cases}$$

$$\Delta x \approx 0.030$$

$$\Delta y \approx 0.000$$

Новое приближение:

$$x_1 = x_0 + \Delta x = -0.900 + 0.030 = -0.870$$

$$y_1 = y_0 + \Delta y = -0.400 + 0.000 = -0.400$$

Вторая итерация:

$$x_1 \approx -0.870, \quad y_1 \approx -0.400$$

Подставляем в систему:

$$\begin{cases} 1.112\Delta x - 1.368\Delta y = 0.001 \\ -1.566\Delta x - 1.598\Delta y = -0.001 \end{cases}$$

Решение системы:

$$\Delta x \approx 0.001$$

$$\Delta y \approx 0.000$$

Новое приближение:

$$x_2 = x_1 + \Delta x \approx -0.870 + 0.001 \approx -0.869$$

$$y_2 = y_1 + \Delta y \approx -0.400 + 0.000 \approx -0.400$$

Так как $|x_2 - x_1| = 0.001 < 0.01$, также как и $|y_2 - y_1| = 0.000 < 0.01$, то мы достигли необходимой точности, значит корень $A(-0.869, -0.400)$

1.2.4.2 Теперь возьмём приближение $x_0 = -0.3, y_0 = 0.7$:

На первой итерации система будет иметь вид:

$$\begin{cases} 1.306\Delta x - 0.302\Delta y = 0.000 \\ -0.540\Delta x + 2.800\Delta y = -0.061 \end{cases}$$

Решаем систему:

$$\begin{cases} 1.306\Delta x - 0.302\Delta y = 0.000 \\ -0.540\Delta x + 2.800\Delta y = -0.061 \end{cases}$$

$$\Delta x \approx -0.005$$

$$\Delta y \approx -0.023$$

Новое приближение:

$$x_1 = x_0 + \Delta x = -0.300 - 0.005 = -0.305$$

$$y_1 = y_0 + \Delta y = 0.700 - 0.023 = 0.677$$

Вторая итерация:

$$x_1 \approx -0.305, \quad y_1 \approx 0.677$$

Подставляем в систему:

$$\begin{cases} 1.294\Delta x - 0.308\Delta y = 0.000 \\ -0.550\Delta x + 2.709\Delta y = -0.001 \end{cases}$$

Решение системы:

$$\Delta x \approx 0.000$$

$$\Delta y \approx 0.000$$

Новое приближение:

$$x_2 = x_1 + \Delta x \approx -0.305 + 0.000 \approx -0.305$$

$$y_2 = y_1 + \Delta y \approx 0.677 + 0.000 \approx 0.677$$

Так как $|x_2 - x_1| = 0.000 < 0.01$, также как и $|y_2 - y_1| = 0.000 < 0.01$, то мы достигли необходимой точности, значит корень $B(-0.305, 0.677)$

Значит всего 4 точки, являющиеся корнями системы:

$$A(-0.869, -0.400), B(-0.305, 0.677),$$

$$A'(0.869, 0.400), B'(0.305, -0.677),$$

2 Программная реализация задачи

По варианту №5 (табл. 9) в программе реализованы методы 1, 3, 5, 7:

Метод	Назначение
Половинного деления	Решение нелинейного уравнения
Ньютона	Решение нелинейного уравнения
Простой итерации	Решение нелинейного уравнения
Простой итерации	Решение системы нелинейных уравнений

2.1 Рабочие формулы и блок-схемы реализованных методов

2.1.1 Метод половинного деления

Рабочая формула:

$$x_n = \frac{a_n + b_n}{2}$$

Выбор новой половины отрезка: если $f(a_n) \cdot f(x_n) \leq 0$, то корень находится в $[a_n, x_n]$, иначе — в $[x_n, b_n]$.

Критерии окончания: $|b_n - a_n| \leq \varepsilon$ или $|f(x_n)| \leq \varepsilon$.

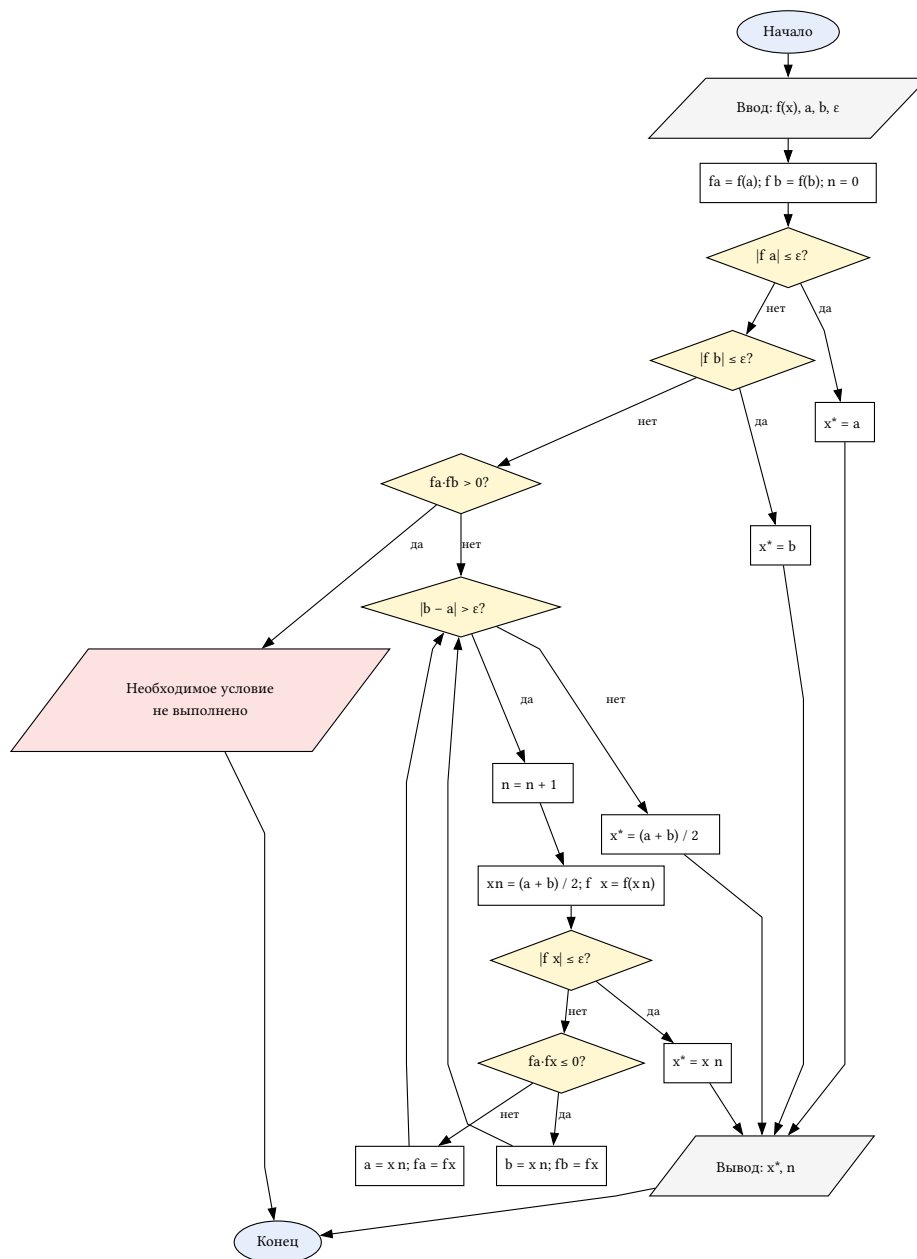


Рис. 1. Блок-схема метода половинного деления

2.1.2 Метод Ньютона (для уравнения)

Рабочая формула:

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f'(x_{n-1})}$$

Начальное приближение x_0 берётся из $[a, b]$ по условию $f(x_0) \cdot f''(x_0) > 0$: в реализации выбирается $x_0 = a$, а если условие не выполнено, $x_0 = b$.

Критерий окончания: $|f(x_n)| \leq \varepsilon$.

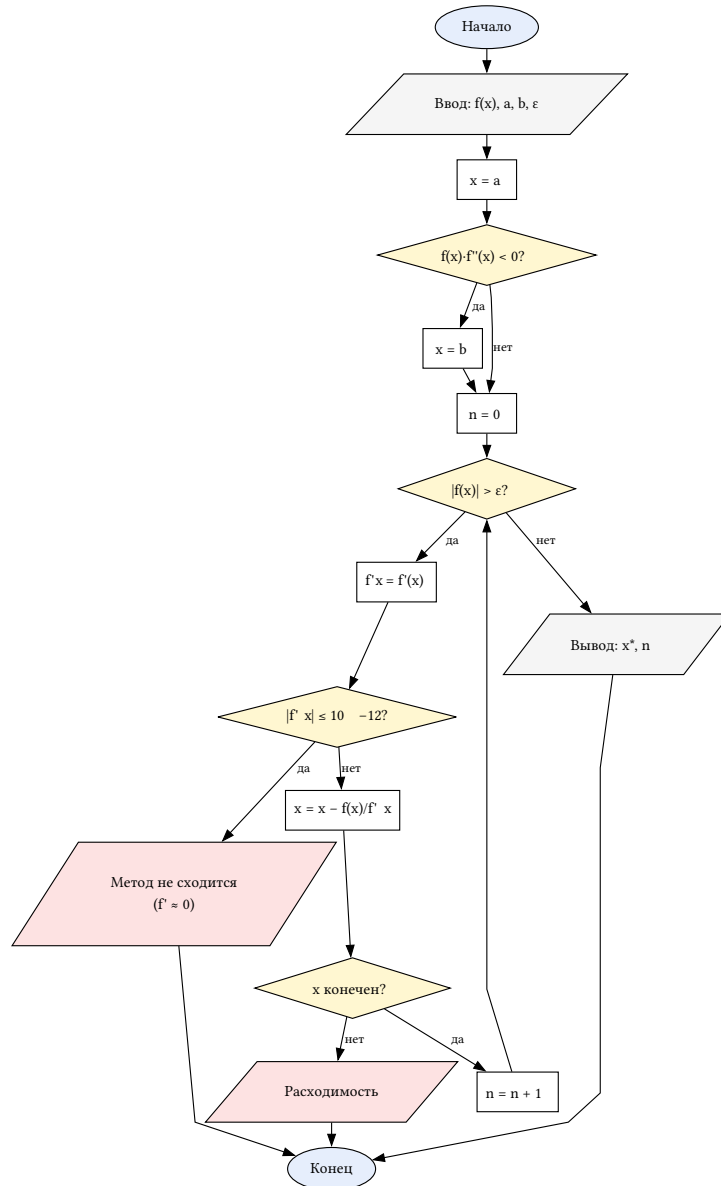


Рис. 2. Блок-схема метода Ньютона

2.1.3 Метод простой итерации (для уравнения)

Исходное уравнение $f(x) = 0$ приводится к виду $x = \varphi(x)$ с помощью вспомогательного параметра λ :

$$\varphi(x) = x + \lambda \cdot f(x), \quad \varphi'(x) = 1 + \lambda \cdot f'(x)$$

$$\lambda = \begin{cases} -1/\max_{x \in [a, b]} |f'(x)|, & \text{если } f'(x) > 0 \text{ на } [a, b] \\ +1/\max_{x \in [a, b]} |f'(x)|, & \text{если } f'(x) < 0 \text{ на } [a, b] \end{cases}$$

Достаточное условие сходимости: $q = \max_{x \in [a, b]} |\varphi'(x)| < 1$.

Начальное приближение выбирается так же, как в методе Ньютона.

Критерий окончания: $|x_{n+1} - x_n| \leq \varepsilon$.

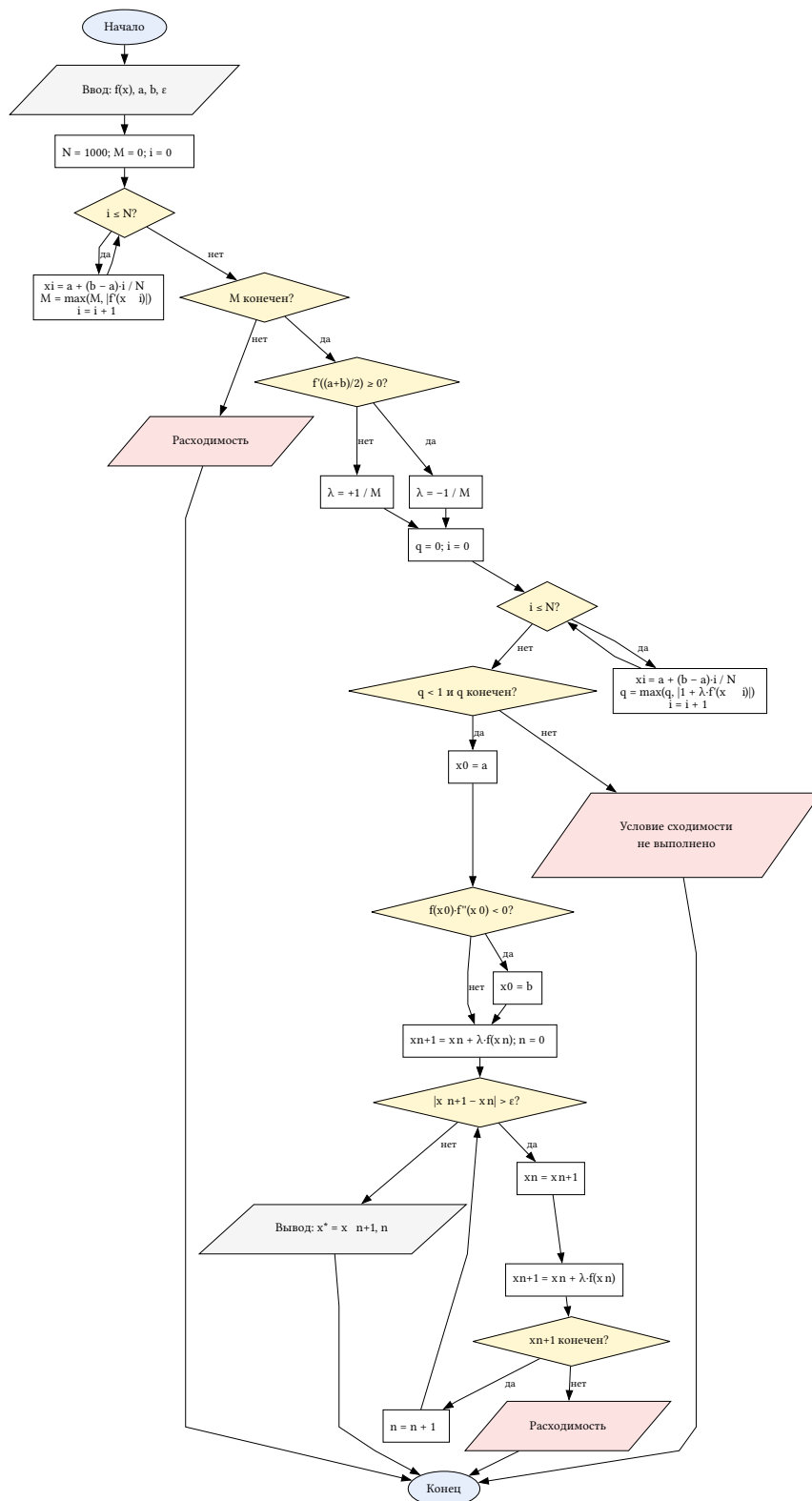


Рис. 3. Блок-схема метода простой итерации для уравнения
 2.1.4 Метод простой итерации (для системы)
 Исходная система приводится к эквивалентному виду:

$$\begin{cases} x = \varphi_1(x, y) \\ y = \varphi_2(x, y) \end{cases}$$

Итерационная формула:

$$x_{k+1} = \varphi_1(x_k, y_k), \quad y_{k+1} = \varphi_2(x_k, y_k)$$

Достаточное условие сходимости (сумма модулей частных производных по строке меньше 1):

$$q = \max \left(\left| \frac{\partial \varphi_1}{\partial x} \right| + \left| \frac{\partial \varphi_1}{\partial y} \right|, \left| \frac{\partial \varphi_2}{\partial x} \right| + \left| \frac{\partial \varphi_2}{\partial y} \right| \right) < 1$$

Критерий окончания: $\max(|x_{k+1} - x_k|, |y_{k+1} - y_k|) \leq \varepsilon$.

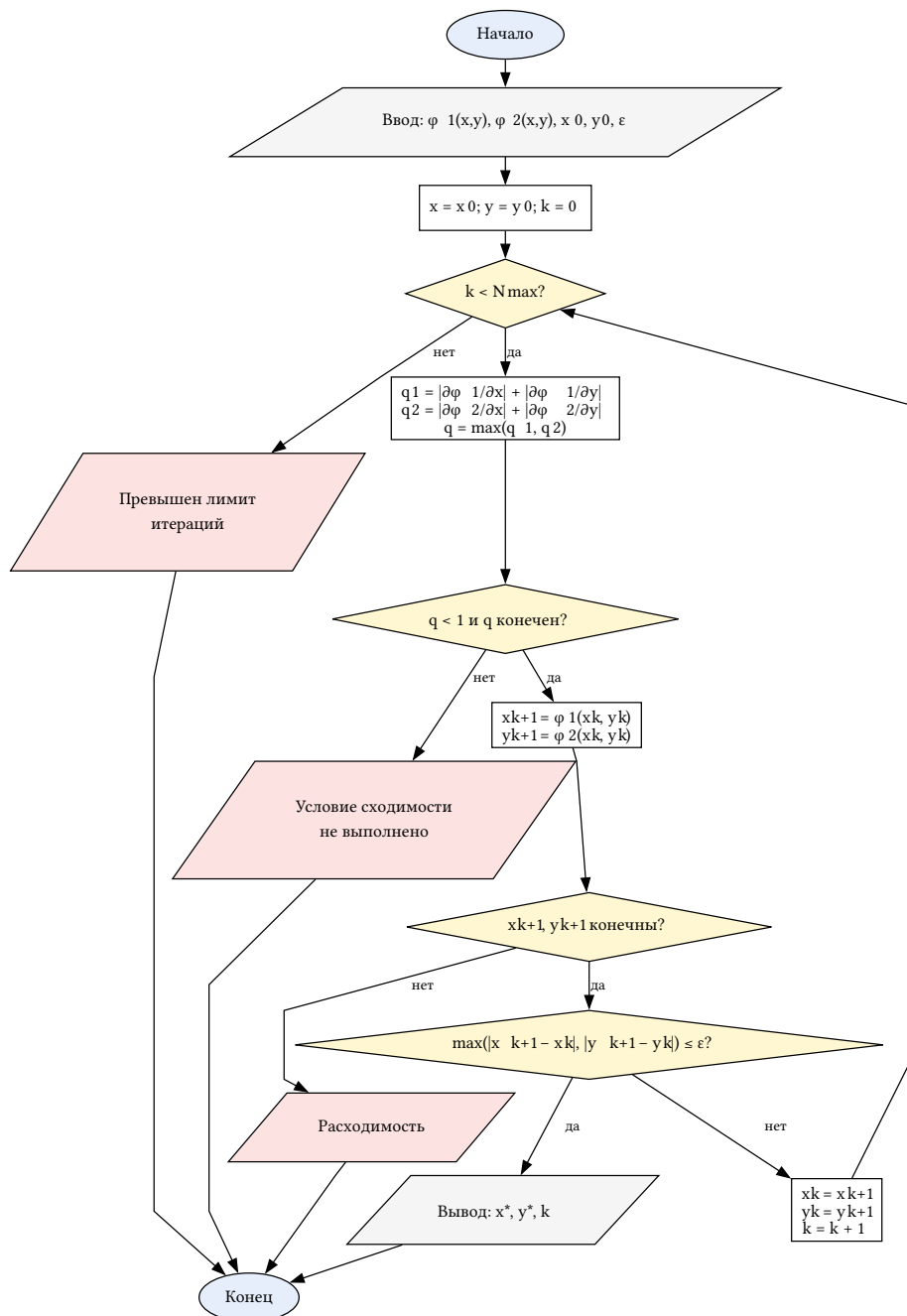


Рис. 4. Блок-схема метода простой итерации для системы

2.2 Исходный код основных элементов расчёта

Заголовочный файл `calc/Solvers.hpp` с описанием базового класса и трёх наследников для нелинейного уравнения, а также класса

`SystemIterSolver` для системы:

```
#pragma once

#include <cmath>
#include <functional>
#include <utility>

using MathFunc = std::function<double(double)>;
using SystemFunc = std::function<double(double, double)>;

enum Status {
    SUCCESS = 0,
    INCORRECT_BORDERS = 1,
    SEVERAL_ROOTS = 2,
    NO_ROOTS = 3,
    INCORRECT_EPS = 4,
    METHOD_DIVERGENCE = 5
};

class Solver {
protected:
    double EPS;

public:
    Solver(double EPS) : EPS(EPS) {};
    virtual ~Solver() = default;
    virtual std::pair<int, double> solve(MathFunc f, double a, double b) = 0;

    static Status validate(MathFunc f, double a, double b, double EPS) {
        if (!(EPS > 0.0) || !std::isfinite(EPS)) {
            return INCORRECT_EPS;
        }

        constexpr int samples = 2000;
        const double step = (b - a) / samples;
        const double zeroTol = 1e-10;

        double prev = f(a);
        if (!std::isfinite(prev)) {
            return INCORRECT_BORDERS;
        }

        int rootCounter = 0;
        bool prevNearZero = std::abs(prev) ≤ zeroTol;
        if (prevNearZero) {
            rootCounter++;
        }

        for (int i = 1; i ≤ samples; ++i) {
            const double x = a + i * step;
            const double cur = f(x);
            if (!std::isfinite(cur)) {
                return INCORRECT_BORDERS;
            }
        }
    }
};
```

```

    }

    const bool curNearZero = std::abs(cur) ≤ zeroTol;
    if ((prev < 0.0 && cur > 0.0) || (prev > 0.0 && cur < 0.0)) {
        rootCounter++;
    } else if (curNearZero && !prevNearZero) {
        rootCounter++;
    }

    prev = cur;
    prevNearZero = curNearZero;
}

if (rootCounter == 0) {
    return NO_ROOTS;
}
if (rootCounter == 1) {
    return SUCCESS;
}
return SEVERAL_ROOTS;
}

double df(MathFunc f, double x);
double d2f(MathFunc f, double x);
};

class DihotomiaSolver : public Solver {
public:
    DihotomiaSolver(double EPS) : Solver(EPS) {};
    std::pair<int, double> solve(MathFunc f, double a, double b) override;
};

class NewtonSolver : public Solver {
public:
    NewtonSolver(double EPS) : Solver(EPS) {};
    std::pair<int, double> solve(MathFunc f, double a, double b) override;
};

class IterSolver : public Solver {
public:
    IterSolver(double EPS) : Solver(EPS) {};
    std::pair<int, double> solve(MathFunc f, double a, double b) override;
};

class SystemIterSolver {
    double EPS;
    static constexpr int kMaxIterations = 10'000;

public:
    explicit SystemIterSolver(double EPS) : EPS(EPS) {}

    std::pair<std::pair<int, double>, std::pair<int, double>>
    solve(SystemFunc phiX, SystemFunc phiY, double x0, double y0) const;

private:
    static double dfdx(SystemFunc f, double x, double y);
    static double dfdy(SystemFunc f, double x, double y);
};

```


Реализация (calc/solver.cpp) — численные производные $df/d2f$, метод половинного деления, метод простой итерации с автоподбором параметра λ , метод Ньютона и метод простой итерации для системы:

```
#include "Solvers.hpp"

#include <algorithm>
#include <cmath>
#include <limits>

double Solver::df(MathFunc f, double x) {
    const double h = 1e-6;
    return (f(x + h) - f(x - h)) / (2 * h);
}

double Solver::d2f(MathFunc f, double x) {
    const double h = 1e-4;
    return (f(x + h) - 2 * f(x) + f(x - h)) / (h * h);
}

std::pair<int, double> DihotomiaSolver::solve(MathFunc f, double a, double b) {
    double fa = f(a);
    double fb = f(b);
    int iter = 0;
    if (std::abs(fa) ≤ EPS) {
        return {0, a};
    }
    if (std::abs(fb) ≤ EPS) {
        return {0, b};
    }
    if (fa * fb > 0.0) {
        return {0, std::numeric_limits<double>::quiet_NaN()};
    }

    while ((b - a) > EPS) {
        iter++;
        const double x_0 = (a + b) / 2.0;
        const double fx = f(x_0);
        if (std::abs(fx) ≤ EPS) {
            return {iter, x_0};
        }
        if (fa * fx ≤ 0.0) {
            b = x_0;
            fb = fx;
        } else {
            a = x_0;
            fa = fx;
        }
    }
    return {iter, (a + b) / 2.0};
}

std::pair<int, double> IterSolver::solve(MathFunc f, double a, double b) {
    constexpr int samples = 1000;
    double maxDf = 0.0;
    for (int i = 0; i ≤ samples; ++i) {
        const double x = a + (b - a) * static_cast<double>(i) / samples;
```

```

    maxDf = std::max(maxDf, std::abs(df(f, x)));
}

if (!std::isfinite(maxDf)) {
    return {0, std::numeric_limits<double>::quiet_NaN()};
}

const double mid = (a + b) / 2.0;
const double dmid = df(f, mid);
const double lambda = (dmid ≥ 0.0 ? -1.0 : 1.0) / maxDf;

double q = 0.0;
for (int i = 0; i ≤ samples; ++i) {
    const double x = a + (b - a) * static_cast<double>(i) / samples;
    q = std::max(q, std::abs(1.0 + lambda * df(f, x)));
}
if (q ≥ 1.0 || !std::isfinite(q)) {
    return {0, std::numeric_limits<double>::quiet_NaN()};
}

double x = a;
if (f(x) * d2f(f, x) < 0.0) {
    x = b;
}

double xn = lambda * f(x) + x;
int iter = 0;
while (std::abs(xn - x) > EPS) {
    x = xn;
    xn = lambda * f(x) + x;
    if (!std::isfinite(xn)) {
        return {iter, std::numeric_limits<double>::quiet_NaN()};
    }
    iter++;
}

return {iter, xn};
}

std::pair<int, double> NewtonSolver::solve(MathFunc f, double a, double b) {
    double x = a;
    if (f(x) * d2f(f, x) < 0.0) {
        x = b;
    }

    int iter = 0;
    while (std::abs(f(x)) > EPS) {
        const double dfx = df(f, x);
        if (std::abs(dfx) ≤ 1e-12) {
            return {iter, std::numeric_limits<double>::quiet_NaN()};
        }
        x = x - f(x) / dfx;
        if (!std::isfinite(x)) {
            return {iter, std::numeric_limits<double>::quiet_NaN()};
        }
        iter++;
    }

    return {iter, x};
}

```

```

double SystemIterSolver::dfdx(SystemFunc f, double x, double y) {
    const double h = 1e-6;
    return (f(x + h, y) - f(x - h, y)) / (2 * h);
}

double SystemIterSolver::dfdy(SystemFunc f, double x, double y) {
    const double h = 1e-6;
    return (f(x, y + h) - f(x, y - h)) / (2 * h);
}

std::pair<std::pair<int, double>, std::pair<int, double>>
SystemIterSolver::solve(SystemFunc phiX, SystemFunc phiY, double x0,
                        double y0) const {
    double x = x0;
    double y = y0;

    for (int iter = 0; iter < SystemIterSolver::kMaxIterations; ++iter) {
        const double q1 = std::abs(dfdx(phiX, x, y)) + std::abs(dfdy(phiX, x, y));
        const double q2 = std::abs(dfdx(phiY, x, y)) + std::abs(dfdy(phiY, x, y));
        const double q = std::max(q1, q2);
        if (!std::isfinite(q) || q ≥ 1.0) {
            return {{iter, std::numeric_limits<double>::quiet_NaN()},
                    {iter, std::numeric_limits<double>::quiet_NaN()}};
        }

        const double nextX = phiX(x, y);
        const double nextY = phiY(x, y);
        if (!std::isfinite(nextX) || !std::isfinite(nextY)) {
            return {{iter, std::numeric_limits<double>::quiet_NaN()},
                    {iter, std::numeric_limits<double>::quiet_NaN()}};
        }

        if (std::max(std::abs(nextX - x), std::abs(nextY - y)) ≤ EPS) {
            return {{iter, nextX}, {iter, nextY}};
        }

        x = nextX;
        y = nextY;
    }

    return {{0, std::numeric_limits<double>::quiet_NaN()},
            {0, std::numeric_limits<double>::quiet_NaN()}};
}

```

2.3 Интерфейс приложения

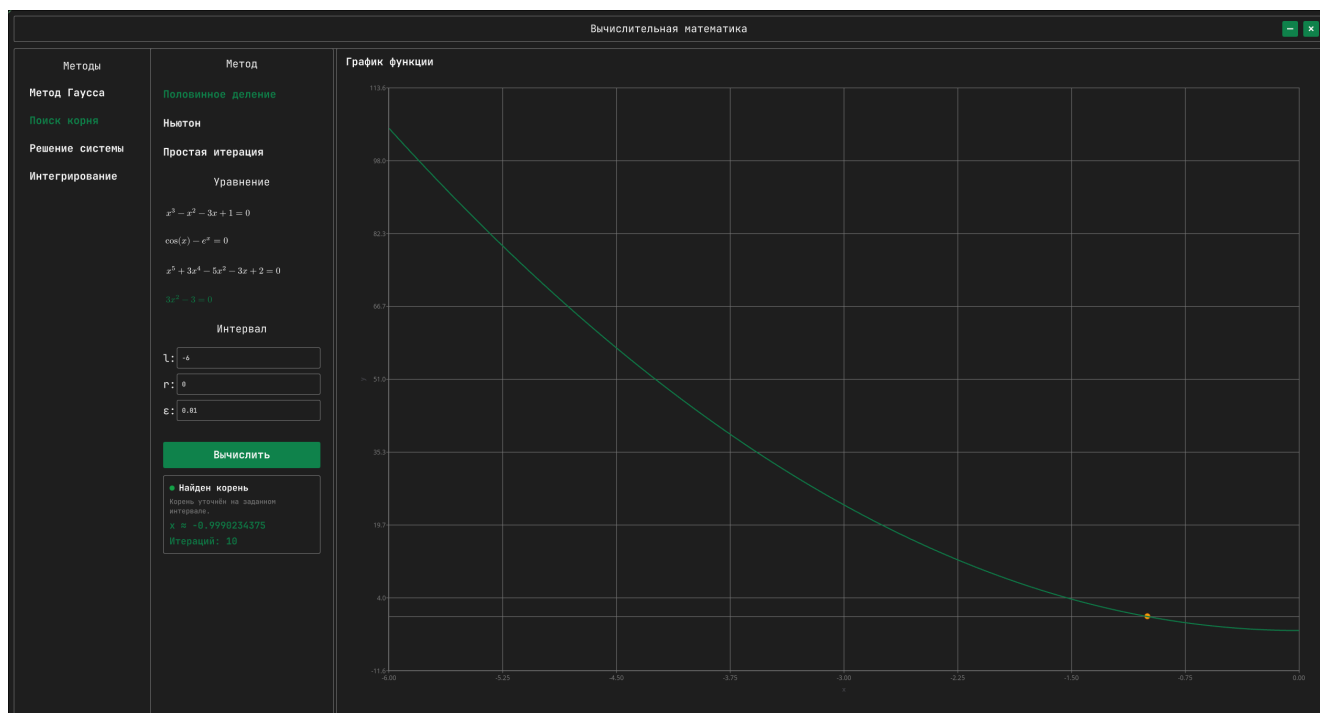


Рис. 5. Внешний вид вкладок «Поиск корня» и «Решение системы» в приложении

3 Выводы

В ходе лабораторной работы изучены и программно реализованы численные методы решения нелинейных уравнений (метод половинного деления, метод Ньютона, метод простой итерации) и систем нелинейных уравнений (метод простой итерации). Для варианта №5 корни уравнения $-2.7x^3 - 1.48x^2 + 19.23x + 6.35 = 0$ уточнены с точностью $\varepsilon = 10^{-2}$ тремя методами, соответствующими расположению каждого корня. Система $\begin{cases} \tan(xy+0.3)=x^2 \\ 0.9x^2+2y^2=1 \end{cases}$ решена методом Ньютона; получены четыре корня, соответствующие точкам пересечения двух кривых. Программная часть покрывает методы 1, 3, 5, 7 (согласно табл. 9 варианта); все методы проверяют условия применимости (наличие и единственность корня, условия сходимости) и возвращают число итераций вместе с приближённым значением корня.